AIAC-13.2 ASSIGNMENT

```python
'''Task Description -1 (Refactoring  Eliminating Repetitive Logic)
• Task: Use AI-assisted coding techniques to restructure a Python
program that performs repeated calculations, transforming it into
a reusable and maintainable design.
Instructions:
• Ask AI to locate repeated computations within the program.
• Encapsulate repeated logic into reusable functions.
• Verify that program output remains unchanged after refactoring.
• Add proper docstrings to all defined functions.
Sample Code:
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
Expected Output:
A refactored Python script that uses a single reusable function to
compute area and perimeter without repeating code.'''
def calculate_rectangle(length, width):
    """
    Function to calculate area and perimeter of a rectangle.
    Parameters:
    length (float): Length of the rectangle
    width (float): Width of the rectangle
    Returns:
    tuple: Area and perimeter of the rectangle
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
# User input
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area, perimeter = calculate_rectangle(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

```python

'''Task Description -2 (Refactoring - Modularizing Inline
Calculations)
• Task: Use AI to refactor a Python script by extracting repeated
inline calculations into reusable and well-documented functions.
Instructions:
• Identify repeated calculation patterns in the code.
• Convert them into reusable functions.
• Ensure readability and proper documentation.
Sample Code:
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
Expected Output:
A modular Python program that uses a reusable function to compute
final amounts for different inputs.
'''
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculate final amount after applying discount.
    Parameters:
    amount (float): Original amount
    discount_rate (float): Discount percentage
    Returns:
    float: Final amount after discount
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount
# User input
amount = float(input("Enter purchase amount: "))
result = calculate_final_amount(amount)
print("Final Amount:", result)


```

```
'''Task Description -3 (Refactoring - Improving Loop and
Conditional Performance)
• Task: Apply AI-based suggestions to enhance the performance of
Python code that relies on nested loops and repeated conditional
checks.
Instructions:
• Use AI to recommend more efficient data structures or logic
simplifications.
• Replace inefficient constructs while preserving program behavior.
• Compare execution efficiency before and after refactoring.
Sample Code:
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
else:
print(name, "not found")
Expected Output:
An optimized Python program that performs faster lookups using
efficient logic, along with execution performance comparison.
'''
import time
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
# BEFORE OPTIMIZATION
start = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
end = time.time()
```

click to restart.

```python
print("Time before optimization:", end - start)
# AFTER OPTIMIZATION USING SET
student_set = set(students)
start = time.time()
for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
end = time.time()
print("Time after optimization:", end - start)



'''Task Description -4 (Refactoring - Replacing Hardcoded Values
with Constants)
• Task: Utilize AI assistance to improve maintainability by
replacing hardcoded numerical values with named constants.
Instructions:
• Detect hardcoded numeric values used directly in expressions.
• Define descriptive constants at the beginning of the program.
• Update calculations to use these constants without changing
results.
Sample Code:z
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
Expected Output:
A revised Python program using constants such as RATE and TIME for
clearer and maintainable calculations..'''
# Constants
RATE = 2
TIME = 5
def calculate_simple_interest(principal):
    """
    Calculate simple interest.
    Parameters:
    principal (float): Principal amount
    Returns:
    float: Simple interest
    """
    return (principal * RATE * TIME) / 100
```

```python
# User input
principal = float(input("Enter principal amount: "))
interest = calculate_simple_interest(principal)
print("Simple Interest:", interest)



'''Task Description -5 (Re factoring - Enhancing Variable Naming
and Code Clarity)
• Task: Apply AI tools to improve code readability by renaming
unclear variables and adding meaningful comments..
Instructions:
• Replace non-descriptive variable names with meaningful
identifiers.
• Add inline comments where the logic may be unclear.
• Ensure program output remains the same.
Sample Code:
x = 50
y = 30
z = x + y
print(z)
Expected Output:
A Python program with clear variable names and explanatory comments
that improve readability without altering functionality'''
# Input values
first_number = float(input("Enter first number: "))
second_number = float(input("Enter second number: "))
# Calculate sum of two numbers
sum_result = first_number + second_number
# Display result
print("Sum of numbers:", sum_result)


```

ALGORITHMS:

Here are the **algorithms for all 5 tasks** written in **clear step-by-step format** suitable for **lab records or assignments**.

Task 1 – Algorithm: Eliminating Repetitive Logic

Algorithm

1. Start the program.
2. Define a function `calculate_rectangle(length, width)`.
3. Inside the function, calculate **area = length × width**.
4. Calculate **perimeter = 2 × (length + width)**.
5. Return the area and perimeter.
6. Ask the user to enter the **length**.
7. Ask the user to enter the **width**.
8. Call the function `calculate_rectangle(length, width)`.
9. Store the returned values.
10. Display the **area and perimeter**.
11. Stop the program.

Task 2 – Algorithm: Modularizing Inline Calculations

Algorithm

1. Start the program.
2. Define a function `calculate_final_amount(amount)`.
3. Set the **discount rate = 10% (0.10)**.
4. Calculate **discount = amount × discount_rate**.
5. Calculate **final_amount = amount – discount**.
6. Return the final amount.
7. Ask the user to enter the **purchase amount**.
8. Call the function `calculate_final_amount(amount)`.
9. Store the returned value.
10. Display the **final amount after discount**.
11. Stop the program.

Task 3 – Algorithm: Improving Loop and Conditional Performance

Algorithm

1. Start the program.
2. Create a list called **students** containing student names.
3. Create another list called **check_list** containing names to search.
4. Convert the **students list into a set** for faster lookup.
5. Start a loop for each name in **check list**.

```
# **Task 3 – Algorithm: Improving Loop and Conditional Performance**
### **Algorithm**
1. Start the program.
2. Create a list called **students** containing student names.
3. Create another list called **check_list** containing names to search.
4. Convert the **students list into a set** for faster lookup.
5. Start a loop for each name in **check_list**.
6. Check if the name exists in the **student set**.
7. If the name exists, display **"name found"**.
8. Otherwise display **"name not found"**.
9. Repeat until all names are checked.
10. Stop the program.
---

# **Task 4 – Algorithm: Replacing Hardcoded Values with Constants**
### **Algorithm**
1. Start the program.
2. Define constants **RATE = 2** and **TIME = 5**.
3. Define a function `calculate_simple_interest(principal)`.
4. Inside the function calculate
   **simple_interest = (principal × RATE × TIME) / 100**.
5. Return the simple interest.
6. Ask the user to enter the **principal amount**.
7. Call the function `calculate_simple_interest(principal)`.
8. Store the returned value.
9. Display the **simple interest**.
10. Stop the program.
---

# **Task 5 – Algorithm: Enhancing Variable Naming and Code Clarity**
### **Algorithm**
1. Start the program.
2. Ask the user to enter the **first number**.
3. Ask the user to enter the **second number**.
4. Store them in meaningful variables `first_number` and `second_number`.
5. Calculate **sum_result = first_number + second_number**.
6. Display the **sum of the numbers**.
7. Stop the program.
'''
```

ANALYSIS:

Task 1 – Eliminating Repetitive Logic
The repeated calculations for area and perimeter were refactored into a reusable function to avoid code duplication.
This improves code readability, maintainability, and reusability.

Task 2 – Modularizing Inline Calculations
The repeated discount calculation was extracted into a function to make the code modular and organized.
This allows easy modification of the discount logic without changing multiple parts of the program.

Task 3 – Improving Loop and Conditional Performance
The nested loop used for searching was optimized using a set data structure for faster lookups.
This reduces time complexity and improves program performance.

Task 4 – Replacing Hardcoded Values with Constants
Hardcoded numerical values were replaced with named constants to make the program easier to understand.
This improves maintainability since changes can be made in one place.

Task 5 – Enhancing Variable Naming and Code Clarity
Unclear variable names were replaced with meaningful names and comments were added.
This improves readability and makes the program easier for developers to understand.

OUTPUT:

```
PS C:\Users\hp\Desktop\AIAC>

PS C:\Users\hp\Desktop\AIAC>
PS C:\Users\hp\Desktop\AIAC> python A132.py
Enter length: 8
Enter width: 4
Area: 32.0
Perimeter: 24.0
Enter purchase amount: 2000
Final Amount: 1800.0
Charan found
Esha not found
Bhavya found
Time before optimization: 0.0007529258728027344
Charan found
Esha not found
Bhavya found
Time after optimization: 0.00079584121704101 56
Enter principal amount: 1000
Simple Interest: 100.0
Enter first number: 50
Enter second number: 30
Enter principal amount: 1000
Simple Interest: 100.0
Enter first number: 50
Enter second number: 30
Simple Interest: 100.0
Enter first number: 50
Enter second number: 30
Enter first number: 50
Enter second number: 30
Enter second number: 30
Sum of numbers: 80.0
PS C:\Users\hp\Desktop\AIAC>
```

Humera Nuzhat

2303A52083

Batch 39